Experiment No. 5

Title: To Develop a Distributed Application Using Messaging System in Publish-Subscribe Paradigm

1. Aim

To develop a messaging-based application using the Publish-Subscribe (Pub-Sub) paradigm and demonstrate concurrent communication between a Publisher and multiple Subscribers using OpenMP.

2. Objectives

After completing this experiment, students will be able to:

1. Understand the Publish-Subscribe messaging paradigm.
2. Understand the role of a Publisher, Topic, and Subscriber.
3. Implement parallel execution using OpenMP.
4. Use OpenMP parallel sections for concurrent execution.
5. Use critical sections to protect shared resources.
6. Demonstrate communication between multiple subscribers through a shared topic.
7. Understand synchronization in parallel applications.

3. Prerequisite Knowledge

Students should have basic knowledge of:

- C programming
- Functions and arrays
- Strings in C
- Process and thread concepts
- Parallel computing
- OpenMP
- Shared-memory programming
- Critical sections and synchronization

4. Theory

4.1 Messaging System

A messaging system allows different components of an application to communicate by exchanging messages.

Instead of one component directly calling another component, a message is sent through a messaging mechanism.

For example:

Application A

|

| Message

↓

Messaging System

|

↓

Application B

Messaging systems are commonly used in distributed applications because they provide a mechanism for exchanging information between independent components.

5. Publish-Subscribe Paradigm

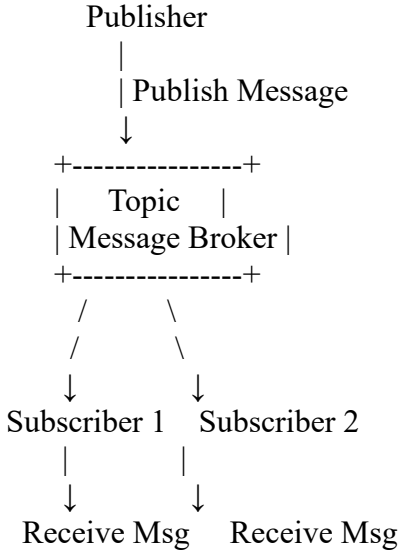
The Publish-Subscribe (Pub-Sub) model is a messaging communication pattern in which:

- A Publisher generates messages.
- A Topic acts as a communication channel.
- A Subscriber receives messages from the topic.

The publisher does not need to know who the subscribers are.

Similarly, subscribers do not need to know who generated the message.

Basic Architecture



6. Real-World Example

Consider a college notification system.

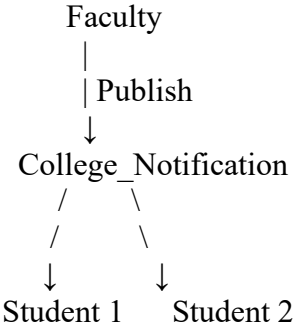
A faculty member publishes:

"PDC Practical Exam will be conducted on Friday."

The message is published to a topic:

College_Notification

Multiple students subscribed to this topic receive the message.



The publisher does not need to send the message separately to each student.

7. Publish-Subscribe vs Point-to-Point

Point-to-Point	Publish-Subscribe
One sender communicates with one receiver	One publisher can communicate with many subscribers
Message generally goes to one consumer	Message can be delivered to multiple subscribers
Direct communication	Topic-based communication
Suitable for one-to-one communication	Suitable for one-to-many communication

Consumer receives a particular message	All interested subscribers receive the message

8. OpenMP
OpenMP (Open Multi-Processing) is an API used for parallel programming in shared-memory systems.
OpenMP allows a program to create multiple threads that can execute tasks concurrently.
Important OpenMP directives used in this experiment are:
#pragma omp parallel sections
and
#pragma omp critical

9. OpenMP parallel sections
The directive:
#pragma omp parallel sections
allows different sections of a program to execute concurrently.
Example:

```
#pragma omp parallel sections
{
   #pragma omp section
   {
      // Publisher
   }

   #pragma omp section
   {
      // Subscriber 1
   }

   #pragma omp section
   {
      // Subscriber 2
   }
}
```

Different sections can be executed by different threads.

10. OpenMP Critical Section
The directive:
#pragma omp critical
ensures that only one thread at a time executes a particular block of code.
Example:

```
#pragma omp critical
{
   messageCount++;
}
```

This is important when multiple threads access or modify shared data.

11. Problem Statement

Develop a C program using OpenMP to implement a simple Publish-Subscribe Messaging System.
The application should:

1. Create a publisher.
2. Create a shared topic.
3. Publish multiple messages.
4. Create at least two subscribers.
5. Allow subscribers to receive the published messages.
6. Use OpenMP for concurrent execution.
7. Use synchronization to protect shared resources.

12. Algorithm
Step 1
Start the program.
Step 2
Create a shared topic to store messages.
Step 3
Create a publisher.
Step 4
Create multiple messages.
Step 5
Publish messages into the shared topic.
Step 6
Create Subscriber 1 and Subscriber 2.
Step 7
Allow subscribers to access the topic concurrently.
Step 8
Display the messages received by each subscriber.
Step 9
Use critical to protect shared data.
Step 10
Terminate the program.

13. Program

```
#include <stdio.h>
#include <omp.h>
#include <string.h>

#define MAX_MESSAGES 5

// Shared topic
char topic[MAX_MESSAGES][100];

// Number of messages published
int messageCount = 0;

int main()
{
   printf("================================================\n");
   printf(" Publish-Subscribe Messaging System\n");
   printf(" Using OpenMP\n");
   printf("================================================\n");

   /*
    * Publisher
    */
   #pragma omp parallel sections
   {
      #pragma omp section
      {
         char *messages[] =
         {
            "PDC Lecture at 9:00 AM",
            "Submit PDC Journal",
            "PDC Assignment Due Today",
            "Practical Exam on Friday",
            "Attend PDC Lecture Regularly"
         };

         for (int i = 0; i < MAX_MESSAGES; i++)
         {
            #pragma omp critical
            {
               strcpy(topic[messageCount],
                     messages[i]);

               messageCount++;

               printf("\nPublisher: %s",
                     messages[i]);
            }
         }
      }
```

```
        /*
         * Subscriber 1
         */
        #pragma omp section
        {
            printf("\nSubscriber 1 is connected to the topic.\n");
        }

        /*
         * Subscriber 2
         */
        #pragma omp section
        {
            printf("Subscriber 2 is connected to the topic.\n");
        }
    }

    /*
     * Subscribers receive messages
     */
    #pragma omp parallel sections
    {
        #pragma omp section
        {
            printf("\n\n========== Subscriber 1 ==========\n");

            for (int i = 0; i < messageCount; i++)
            {
                printf("Subscriber 1 received: %s\n",
                        topic[i]);
            }
        }

        #pragma omp section
        {
            printf("\n========== Subscriber 2 ==========\n");

            for (int i = 0; i < messageCount; i++)
            {
                printf("Subscriber 2 received: %s\n",
                        topic[i]);
            }
        }
    }

    printf("\nAll messages delivered successfully.\n");

    return 0;
}
```

14. Sample Output
The exact ordering of thread-related output may vary.

```
==========================================
 Publish-Subscribe Messaging System
 Using OpenMP
==========================================

Subscriber 1 is connected to the topic.
Subscriber 2 is connected to the topic.

Publisher: PDC Lecture at 9:00 AM
Publisher: Submit PDC Journal
Publisher: PDC Assignment Due Today
Publisher: Practical Exam on Friday
Publisher: Attend PDC Lecture Regularly

========== Subscriber 1 ==========
Subscriber 1 received: PDC Lecture at 9:00 AM
Subscriber 1 received: Submit PDC Journal
Subscriber 1 received: PDC Assignment Due Today
Subscriber 1 received: Practical Exam on Friday
Subscriber 1 received: Attend PDC Lecture Regularly

========== Subscriber 2 ==========
Subscriber 2 received: PDC Lecture at 9:00 AM
Subscriber 2 received: Submit PDC Journal
Subscriber 2 received: PDC Assignment Due Today
Subscriber 2 received: Practical Exam on Friday
Subscriber 2 received: Attend PDC Lecture Regularly

All messages delivered successfully.
```

15. Compilation and Execution
Linux / Ubuntu
Save the program as:
pubsub.c
Compile:

```
gcc -fopenmp pubsub.c -o pubsub
```

Run:

```
./pubsub
```

Windows using GCC/MinGW

```
gcc -fopenmp pubsub.c -o pubsub.exe
```

Run:

```
pubsub.exe
```

16. Advantages of Publish-Subscribe

1. Loose coupling between publisher and subscriber.
2. One message can be delivered to multiple subscribers.
3. Publishers do not need to know subscriber details.
4. Easy to add new subscribers.
5. Suitable for event-driven applications.
6. Supports asynchronous communication.

17. Limitations of This OpenMP Demonstration

Students should understand an important distinction:

OpenMP is primarily a shared-memory parallel programming API, whereas a real distributed Publish-Subscribe system normally uses a message broker and network communication.

This lab program uses OpenMP to demonstrate the Pub-Sub concept and concurrent publisher/subscriber processing, rather than implementing a production distributed messaging broker.

For a true distributed implementation, technologies such as MQTT, Apache Kafka, RabbitMQ, or Java Message Service (JMS) can be used.

18. Conclusion

Thus, a messaging-based application using the Publish-Subscribe paradigm was developed successfully using OpenMP, demonstrating concurrent message publishing and reception by multiple subscribers with appropriate synchronization of shared resources.